Pensamiento computacional

Nombre: _____ Fecha: _____

Qué vas a aprender

- Entender qué es el pensamiento computacional y para qué sirve
- Aplicar sus cuatro pilares: descomposición, patrones, abstracción y algoritmos
- Resolver problemas cotidianos y de programación paso a paso
- Depurar soluciones buscando y corrigiendo errores

Para pensar

¿Qué tienen en común organizar una fiesta de cumpleaños, resolver un cubo de Rubik y programar un videojuego?

Explicación

¿Qué es el pensamiento computacional?

El **pensamiento computacional** es una forma de resolver problemas inspirada en cómo trabajan los informáticos: plantear el problema de manera que una persona o una máquina pueda ejecutar la solución.

No consiste en pensar como un ordenador, sino en organizar nuestras ideas para que un ordenador nos ayude. La investigadora **Jeannette Wing** lo popularizó en 2006 y defendió que debía ser una habilidad básica, como leer, escribir o calcular.

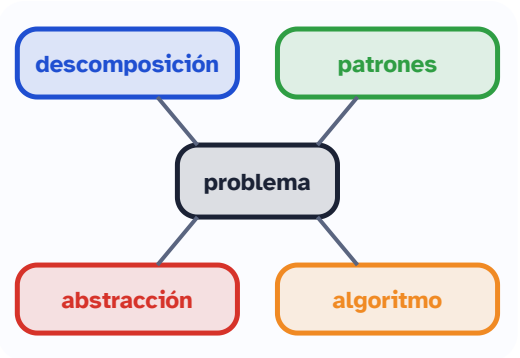
Se apoya en **cuatro pilares**: descomposición, reconocimiento de patrones, abstracción y algoritmos.

undefined

Los cuatro pilares

Pilar	Qué significa	Ejemplo: preparar una mochila de excursión
Descomposición	dividir un problema grande en partes pequeñas	comida, ropa, botiquín, documentos
Patrones	buscar semejanzas o repeticiones	cada día necesito lo mismo: repetir la lista
Abstracción	quedarse con lo esencial e ignorar lo irrelevante	importa el peso y el tiempo, no el color de la mochila
Algoritmo	escribir los pasos ordenados de la solución	1) mirar el tiempo, 2) elegir ropa, 3) meter comida...

Los cuatro se usan juntos y en cualquier orden.



Descomposición

Un problema grande asusta; muchos problemas pequeños, no. **Descomponer** es dividir un problema en subproblemas más sencillos que se pueden resolver por separado (y a veces por personas distintas).

Por ejemplo, programar un videojuego de plataformas se descompone en:

- mover al personaje con el teclado,
- hacer que salte y caiga (gravedad),
- detectar las plataformas,
- contar puntos y vidas,
- mostrar la pantalla de fin de partida.

Cada parte puede volver a dividirse hasta que sea fácil de programar. Así trabajan los equipos de software: cada persona se encarga de un módulo.

undefined

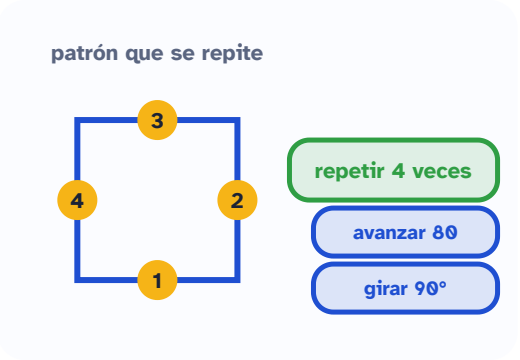
Reconocimiento de patrones

Un **patrón** es algo que se repite. Detectarlo ahorra trabajo: si ya sabemos resolver un caso, podemos reutilizar la solución en los parecidos.

- En matemáticas: 2, 4, 6, 8… cada número es el anterior más 2.
- En un dibujo: un cuadrado se dibuja repitiendo 4 veces «avanza y gira 90°».
- En un juego: todos los enemigos se mueven igual, solo cambia su posición inicial.

En programación, los patrones se convierten en bucles (repetir) y en funciones (bloques reutilizables). Detectar patrones es lo que permite que un programa

corto haga mucho trabajo.



Abstracción

Abstraer es quedarse con la información importante para resolver el problema y descartar los detalles que no influyen. Es una de las ideas más poderosas de la informática.

El ejemplo clásico es el **plano del metro**: no respeta las distancias reales ni las curvas de las vías. Solo muestra lo que necesita el viajero: estaciones, líneas y conexiones. Por eso es tan fácil de usar.

Al programar un juego de fútbol, no necesitamos simular cada músculo del jugador: basta con su posición, su velocidad y si tiene el balón. Elegir bien qué datos guardar es la base de un buen programa.

undefined

Algoritmos

Un **algoritmo** es una secuencia finita, ordenada y precisa de instrucciones para resolver un problema. Una receta de cocina, las instrucciones de un mueble o el camino para llegar a clase son algoritmos.

Para que un ordenador lo ejecute, el algoritmo debe ser:

- **Preciso**: sin ambigüedades. «Echa un poco de sal» no sirve para una máquina; «añade 5 g de sal», sí.
- **Ordenado**: los pasos tienen una secuencia.
- **Finito**: termina en algún momento.
- **Con entrada y salida**: recibe datos y produce un resultado.

En el próximo tema aprenderás a representarlos con diagramas de flujo.

undefined

Depurar: aprender de los errores

Casi ningún programa funciona a la primera. **Depurar** (en inglés, *debugging*) es buscar y corregir los errores, llamados **bugs**. Se cuenta que el término se popularizó cuando la programadora **Grace Hopper** y su equipo encontraron una polilla atascada en un ordenador en 1947.

Estrategias para depurar:

- Ejecutar el programa **paso a paso** y comprobar qué hace en cada momento.
- Probar con datos distintos, incluidos casos extremos (`0`, números negativos, listas vacías).
- Cambiar **una sola cosa** cada vez.
- Explicar el código en voz alta a otra persona (o a un patito de goma).

undefined

Pensamiento computacional sin ordenador

Estas habilidades se pueden practicar **desenchufadas**, sin pantallas:

- **Robot humano**: un compañero da órdenes precisas («avanza 2 pasos», «gira a la derecha») para que otro llegue a un objetivo con los ojos tapados.
- **Cuadrícula de colores**: codificar un dibujo píxel a píxel con números y que otro lo reconstruya.
- **Ordenar**: buscar el método más rápido para ordenar cartas de menor a mayor.
- **Juegos de lógica**: sudokus, laberintos, el juego del Nim.

Así se entiende que el pensamiento computacional es una manera de pensar, no una tecnología.

undefined

Para qué sirve en la vida real

El pensamiento computacional está detrás de casi todo lo digital: el GPS que calcula la ruta más corta, las recomendaciones de series, la predicción del tiempo o los robots que ayudan en un quirófano.

Pero también es útil sin ordenador: organizar el estudio de un examen (descomponer el temario), detectar errores que repites en los ejercicios (patrones), resumir un tema (abstracción) o planificar una tarea en grupo (algoritmo). Por eso se considera una competencia clave del siglo XXI y forma parte de la **competencia digital** del currículo.

undefined

Los cuatro pilares y la depuración

Descomposición	dividir el problema en partes pequeñas
Patrones	buscar lo que se repite → bucles y funciones
Abstracción	quedarse con lo esencial (plano del metro)
Algoritmo	pasos finitos, ordenados y precisos
Depuración	probar, encontrar el bug y corregirlo

Ejercicios

1. **Define los cuatro pilares del pensamiento computacional.**

2. **Descompón el problema «organizar una excursión de clase».**

3. **Encuentra el patrón: 3, 7, 11, 15… ¿Qué número sigue?**

4. **¿Qué información necesitas para programar un semáforo? ¿Qué puedes ignorar?**

5. **Reescribe de forma precisa: «calienta la leche un rato».**

6. **¿Qué ángulo se gira en cada vértice para dibujar un triángulo equilátero con «avanza y gira»?**

7. **Un programa suma bien 2 + 3 pero falla con 0 + 0. ¿Qué estrategia de depuración se aplicó?**

8. **Explica por qué el plano del metro es una abstracción.**

9. **Escribe el algoritmo para lavarte los dientes en 6 pasos.**

10. **Inventa una actividad desenchufada para enseñar bucles a niños de primaria.**

Practica

1. **Dividir un problema grande en partes pequeñas es...**

   A) abstracción
   B) descomposición
   C) depuración

2. **Quedarse con lo esencial e ignorar detalles irrelevantes es...**

   A) patrón
   B) algoritmo
   C) abstracción

3. **Buscar lo que se repite corresponde al pilar de...**

   A) reconocimiento de patrones
   B) descomposición
   C) depuración

4. **Un algoritmo debe ser...**

   A) largo y complejo
   B) escrito en inglés
   C) finito, ordenado y preciso

5. **«Echa un poco de sal» no es una buena instrucción para una máquina porque...**

   A) es muy larga
   B) es ambigua
   C) no tiene números romanos

6. **Buscar y corregir errores en un programa se llama...**

   A) depurar
   B) compilar
   C) abstraer

7. **¿Quién popularizó el término pensamiento computacional en 2006?**

   A) Grace Hopper
   B) Jeannette Wing
   C) Ada Lovelace

8. **Repetir «avanza y gira 90°» cuatro veces dibuja un...**

A) triángulo
B) círculo
C) cuadrado

9. **Una actividad desenchufada es la que se hace…**

A) sin ordenador
B) sin internet pero con tablet
C) con robots

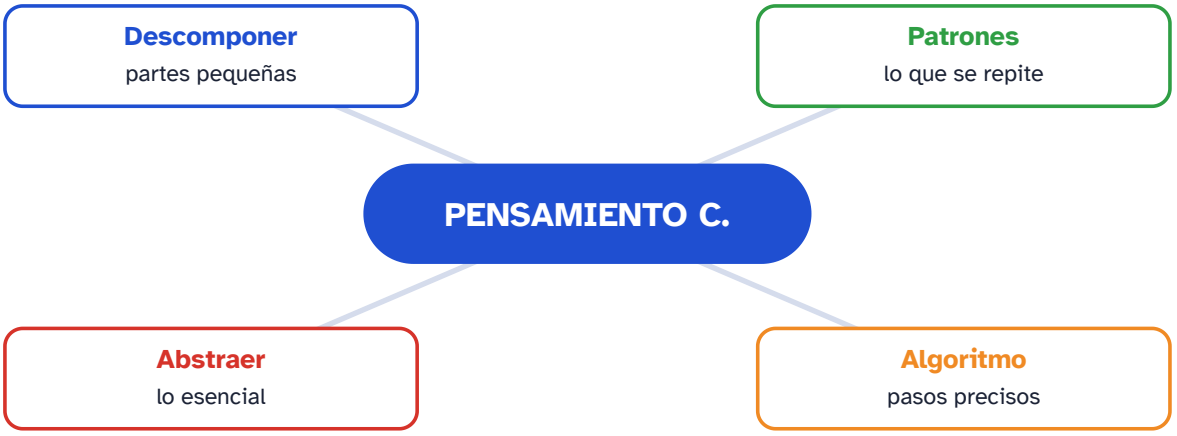
10. **Al depurar, conviene cambiar…**

A) todo a la vez
B) una sola cosa cada vez
C) nada

Explícalo con tus palabras

Incluye: Qué es el pensamiento computacional y por qué no es lo mismo que programar; Los cuatro pilares con un ejemplo de tu vida; Qué características debe tener un algoritmo; Cómo depurar un error.

Resumen



Tarjetas de repaso

Recórtalas: pregunta delante, respuesta detrás.

Pensamiento computacional

Resolver problemas de forma que una persona o máquina pueda ejecutar la solución.

Descomposición

Dividir un problema en subproblemas.

Patrón

Algo que se repite y permite reutilizar soluciones.

Abstracción

Quedarse con lo importante y descartar lo irrelevante.

Algoritmo

Secuencia finita, ordenada y precisa de instrucciones.

Bug

Error en un programa.

Depurar

Buscar y corregir errores.

Actividad desenchufada

Practicar el pensamiento computacional sin ordenador.

Soluciones

1. **B)** Primer pilar.
2. **C)** Como el plano del metro.
3. **A)** Lleva a bucles y funciones.
4. **C)** Para que una máquina lo ejecute.
5. **B)** Falta precisión.
6. **A)** Debugging.
7. **B)** Investigadora estadounidense.
8. **C)** Patrón convertido en bucle.
9. **A)** Papel, juego, movimiento.
10. **B)** Así sabes qué lo arregló.

Soluciones de los ejercicios

1. Descomposición (dividir), patrones (lo que se repite), abstracción (lo esencial) y algoritmos (pasos ordenados).
2. Por ejemplo: destino, transporte, presupuesto, autorizaciones, comida, horario.
3. Se suma 4 cada vez: 19.
4. Necesitas: colores, orden y duración de cada luz. Puedes ignorar: el material, la altura o la marca.
5. Por ejemplo: «calienta 200 ml de leche en el microondas a 800 W durante 60 segundos».
6. 360° / 3 = 120°.
7. Probar con casos extremos.
8. Porque elimina distancias y curvas reales y muestra solo estaciones, líneas y conexiones, que es lo que necesita el viajero.
9. Ejemplo: coger cepillo, poner pasta, cepillar 2 minutos, escupir, enjuagar, guardar cepillo.
10. Respuesta abierta; por ejemplo, una coreografía con un paso que se repite N veces.